Program Optimization for Verifiable Problems

STAT 4830 - Final Report
Spring 2026

Allen Liu · Winson Wang · Jonathan Xu

Abstract and Motivation

This report documents our semester-long exploration of AI-driven program optimization on strictly verifiable problems. Motivated by the observation that LLMs excel at generating solutions but struggle to verify them, we placed LLMs inside deterministic evaluation loops to iteratively improve solutions across four distinct domains: (1) gradient-based RL for circle packing and autocorrelation minimization, (2) prompt/program optimization with GEPA for circle packing and the AirBench CIFAR-10 training benchmark, (3) Parameter Golf — training a ≤16 MB language model in ≤10 minutes on 8×H100s, and (4) Vector Database benchmarking via LLM-driven Rust code optimization.

Across these diverse benchmarks and tasks, we hope to demonstrate evidence that LLMs are capable of solving hard tasks (often harder than a human baseline can complete) given the right environment and setup to succeed.

1. Motivation

Large language models are very good at generating candidate solutions but relatively bad at verifying them.[1] The generation-verification gap motivates a different paradigm: place LLMs inside a loop with a deterministic evaluator and have them iteratively beat the current best record until a new state of the art is achieved. This approach was first pioneered by AlphaEvolve, a paper released by Google in 2025, and later popularized by Andrej Karpathy's repository "autoresearch." Fundamentally, the idea between the two is the same, in particular to remove human supervision from the model's problem solving process and let the model iterate on its own. This new approach has seen remarkable results in algorithm engineering, combinatorics, and systems code optimization, and forms the conceptual spine of this project.

Many real-world tasks are strictly verifiable, meaning the process of evaluation is cheap, fast, and unambiguous. For example:

- Kernel speed: does this GPU kernel run faster than the incumbent?
- Circle packing: does this configuration achieve a higher sum of radii?
- Coding benchmarks: does this model reach ≥94% accuracy faster?
- Language model compression: does this 16 MB model achieve lower BPB?
- Vector database throughput: does this Rust implementation achieve higher QPS at ≥0.95 recall?

This report documents our semester-long exploration of AI-driven program optimization on strictly verifiable problems. We placed LLMs inside deterministic evaluation loops to iteratively improve solutions across four distinct domains:

(1) Gradient-based RL (TTT-Discover): fine-tuning gpt-oss-120b with LoRA adapters at test time, guided by deterministic reward from circle packing (CP26) and autocorrelation minimization (AC1) evaluators.
(2) Prompt/program optimization (GEPA): using a reflection LLM to propose edits to full solver programs, evaluated on CP26 circle packing and the AirBench CIFAR-10 training benchmark, without any weight training.
(3) Parameter Golf: training a ≤16 MB language model in ≤10 minutes on 8×H100 GPUs, evaluated on FineWeb validation BPB, using shallow recurrence, quantization, and architectural innovations.
(4) Vector Database Benchmarking: using an LLM-driven Rust code optimization loop powered by (originally) Qwen and (later) Codex to iteratively improve QPS on a vector similarity search benchmark under a hard recall ≥ 0.95 constraint.

[1] Jason Wei, Asymmetry of verification and verifier's rule, 2025.

2. Part I: Gradient Optimization (TTT-Discover)

2.1 Problem Statement

We studied how test-time RL fine-tuning of a large language model (gpt-oss-120b with LoRA rank-32 adapters) can improve AI-driven search on strictly verifiable optimization problems. Two benchmarks were used, both drawn from the AlphaEvolve task suite:

- CP26 (Circle Packing, N=26): Place 26 circles in $[0,1]^2$ maximizing the sum of radii. Target: sum ≥ 2.636 (SOTA: 2.635983). The model generates Python code that constructs a packing, and an independent verifier recomputes feasibility and score from scratch.
- AC1 (Autocorrelation Inequality): Minimize the bound $C_1 = 2n \cdot \max(\text{conv}(f,f))/(\text{sum}(f))^2$. Target: $C_1 \leq 1.5030$. The model likewise generates Python code, which is executed and scored deterministically.

2.2 Technical Approach: TTT-Discover

Each problem was cast as a Markov Decision Process (MDP): the LLM receives a prompt containing the problem specification and the current best solution, generates Python code, and the code is evaluated deterministically. The reward is a scalar from that evaluation. For CP26, reward is the verified sum of radii when the packing is valid and 0 otherwise; for AC1, reward is derived from the evaluated bound, with invalid generations assigned zero reward. LoRA adapter weights are updated after each batch of evaluated samples (lr=4e-5, 64 samples/batch, 8 groups of 8). A PUCT buffer[2] accumulates high-scoring solutions and biases future sampling toward promising regions.

The system used a split-compute design: model inference and LoRA updates ran remotely through the Tinker API,while deterministic evaluation ran locally on CPU and was parallelized with Ray. This mattered because AC1 evaluations were substantially slower than CP26 evaluations.

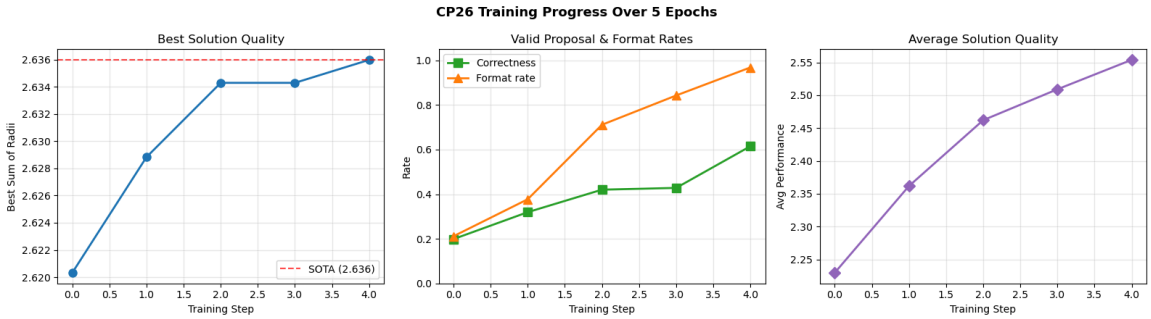
Key hyperparameters:

Parameter	Value	Description
Model	gpt-oss-120b	Base LLM (Tinker API)
LoRA rank	32	Low-rank adapter rank
Learning rate	4e-5	LoRA weight update
Batch size	64	Samples per training step
Group size	8	Trajectories per group for advantage estimation
Advantage estimator	Entropic adaptive beta	KL-regularized
Sampler	PUCT with backprop	Tree-search state selection
Epochs	5	Initial budget

[2] Christopher D. Rosin. 2011. "Multi-armed bandits with episode context." *Annals of Mathematics and Artificial Intelligence* 61(3): 203–230.

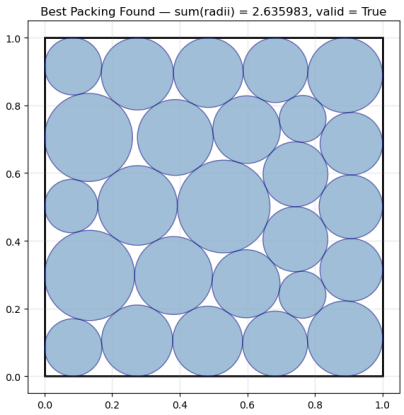
2.3 Results

CP26 results across 5 training epochs:



Step	Correctness	Format Rate	Avg Perf.	Best Score	Wall Time
0	19.9%	21.2%	2.2297	2.6203	30 min
1	31.9%	37.7%	2.3618	2.6288	33 min
2	42.0%	71.1%	2.4620	2.6343	28 min
3	42.8%	84.3%	2.5091	2.6343	39 min
4	61.5%	96.8%	2.5539	2.6360 ✓	24 min

CP26 matched the known SOTA (2.636) within 5 epochs and 2.6 hours. Correctness tripled (19.9% → 61.5%) and format compliance improved 4.6× (21.2% → 96.8%). frac_mixed = 1.0 throughout, confirming strong contrastive RL signal. The saved best construction reached an exact verified sum of radii of 2.635983084887735 (shown on the right), effectively matching the benchmark frontier and exceeding the performance reported on this benchmark by AlphaEvolve originally.



AC1 results:

Step	Correctness	Format Rate	Best C_1	Wall Time
0	23.0%	26.7%	1.5199	63 min
1	25.5%	28.4%	1.5180	40 min
2	30.2%	38.6%	1.5180	38 min
3	37.1%	45.9%	1.5249	64 min
4	38.1%	51.9%	1.5146	32 min

AC1 did not reach its target (best C_1 = 1.5146, gap of 0.77% from ≤1.5030). Correctness improved only 1.7× vs. 3.1× for CP26, and format compliance plateaued at 51.9%. The reward max was remarkably stable (~0.659), suggesting a strong local optimum. It also consumed substantially more evaluation time than CP26, making it both less effective and more expensive under the same 5-epoch budget.

2.4 Key Insights

- RL-based test-time training works: CP26 reached SOTA in 5 epochs from a cold-start model.
- The model learns syntax before semantics: format compliance improves first, then correctness, then quality.
- Task selection matters: CP26's smoother solution space and shorter eval timeout enabled 3× faster learning than AC1.
- ~80% of compute is wasted in early epochs when proposal validity is low — better prompts would have outsized impact.
- Context (PUCT buffer of good solutions) may matter as much as LoRA weight updates.
- Sparse reward program optimization becomes much easier once the model consistently emits runnable, checker-compatible code. Validity matters, not just objective quality.
- Upweighting best solutions exponentially and using large models are critical; small models cannot learn effectively from sparse rewards.

3. Part II: Prompt/Program Optimization (GEPA)

3.1 What is GEPA?

GEPA (Generalized Evolutionary Program Architect) uses a reflection LLM to propose edits to a solver program. The solver is executed locally and may call an LLM once per run to query a strategy. A deterministic evaluator verifies constraints and scores the output. Unlike TTT-Discover, GEPA avoids training weights entirely — it performs structured search over programs guided by verifier feedback.

Why GEPA over gradient-based RL?

- Avoids expensive weight updates; uses cheaper LLMs (Gemini 2.5 Flash Lite, o3-mini, etc.).
- Structured program search is interpretable and debuggable.
- Directly composable with multi-agent architectures.
- Produces auditable artifacts at the program level: candidate source files, per-candidate evaluation JSON, milestone logs, and proposal traces.
- Lets us optimize complete solver programs directly, rather than only prompts or model weights.

3.2 Circle Packing with GEPA (Weeks 6–8)

CP26 (Circle Packing, N=26) is the problem of placing 26 non-overlapping circles inside the unit square $[0,1]^2$ such that the sum of their radii is maximized. Each circle must fit fully within the square, and no two circles may overlap. The known state-of-the-art sum of radii is roughly 2.635983, making it a well-defined benchmark with a precise numerical target.

We chose CP26 as our primary test bed for several reasons. First, it is strictly verifiable: given any proposed configuration, a deterministic geometric checker can instantly confirm whether all non-overlap and boundary constraints are satisfied and compute an exact score with no ambiguity. Second, the evaluation is fast: checking a 26-circle packing takes milliseconds of CPU time, which means an LLM-in-the-loop system can generate and score hundreds of candidates per hour. Third, the task is a natural fit for LLM-generated code: the model outputs a Python script that computes a configuration and prints JSON, so the LLM is reasoning about algorithmic strategy (gradient descent, SciPy optimization, random restarts) rather than generating raw numerical coordinates. Finally, CP26 has a well-known SOTA target (2.636), which gives us a clear finish line to race toward and a meaningful way to compare approaches — we either reach SOTA or we don't.

We applied GEPA to CP26 circle packing using its *optimize_anything* interface in Agent Architecture Evolution mode. The candidate unit was a full Python solver program, evaluated locally via subprocess execution with a hard timeout and an independent verifier that recomputed feasibility and score. The output contract required valid JSON

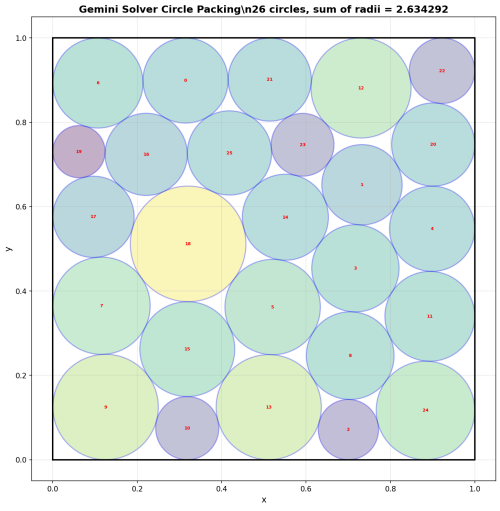
with exactly 26 circles and a claimed sum_radii, but the evaluator never trusted the claimed score without recomputation.

Results by reflection model:

Run	Reflection LLM	Best Score	Target	Notes
A	OpenAI o3-mini	2.4699	2.636	Reached 93.7% of target; stable but plateaued lower.
B	Gemini 2.5 Flash	2.6360	2.636	Matched SOTA by iteration 21; halted by free-tier rate limit.
C	Gemini 2.5 Flash Lite	2.6343	2.636	Near-SOTA when seeding/ randomness controlled.

Gemini 2.5 Flash reached near-SOTA (2.636) much faster and cheaper than RL. Model capability dominated: o3-mini plateaued while Flash closed the remaining gap. A key lesson from this phase was that GEPA's effectiveness depended heavily on the reflection model: stronger models not only proposed better optimization ideas, but also produced more contract-compliant code, which greatly reduced wasted evaluations.

Notably, we achieved this using Gemini 2.5 Flash — a model released before the known SOTA value existed, meaning it had no prior knowledge of the best-known solution. It found this result purely through GEPA's iterative improvement loop. The fact that a small, cheap model came this close to the theoretical optimum is precisely the point: strictly verifiable problems let you extract a lot of performance from very little compute, because the evaluation oracle does the heavy lifting.



(To the right is a sample solution outputted by Gemini 2.5 Flash.)

At the same time, the CP26 experiments exposed important weaknesses in the initial implementation: result reporting initially read the wrong GEPA field (best_score instead of best_candidate), rate limits truncated promising runs, and proposal history was too shallow, making it easy for the search to rediscover similar ideas repeatedly.

3.3 AirBench CIFAR-10 with GEPA (Weeks 8–12)

AirBench challenges a complete Python training program to reach ≥94% accuracy on CIFAR-10 as fast as possible. In our pipeline, the optimization target was a full standalone *solver_code* script evaluated remotely on a single NVIDIA A100-40GB via Modal, under a strict CLI/JSON contract. The official reference used as our starting seed was airbench94_muon.py provided in the official AirBench repo, recording 94.01% accuracy in 2.59 s on 8×H100 (0.29 PFLOPs). We evaluated two approaches:

3.3.1 Autoresearch Loop

An iterative loop using Gemini-3.1-flash-lite-preview on an A100-40GB via Modal. An agent proposes modifications, evaluates them, and logs success/failure to memory.md so future proposals can learn from context. Starting from the best AirBench incumbent script, the loop maintained an incumbent-of-record, used proxy evaluation during search, and only promoted candidates after stronger confirmation runs. Below we show that in a 10-iteration cycle, the SOTA was able to be improved from ~2.57 to 2.50 seconds when run on an A100 (not the H100 setup used in the original benchmark). The particular changes that led to this improvement were first enabling TF32 matrix multiplications and then increasing batch size from 2000 to 2500. Throughout the iterations, we noticed the model opted for mainly tweaking parameters rather than rewriting significant chunks of the solution; when set to more aggressive settings in other runs we noticed the rate of compiler issues rose dramatically. Thus we decided to design a new setup where validation is a core component of the iteration process.

3.3.2 GEPA Multi-Agent Loop (Weeks 8–12)

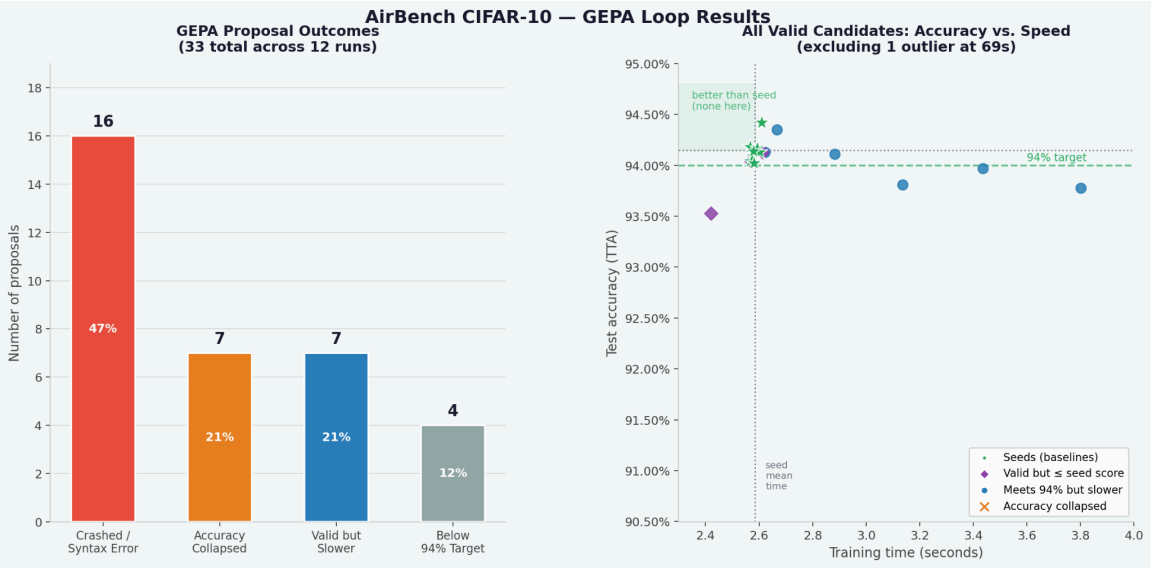
A five-role agent team (Coordinator, Performance Worker, Reliability Worker, Integrator, Reviewer) proposes edits to sections of the program. Each proposal is evaluated end-to-end on Modal A100-40GB. Internally, the program is split into semantic sections such as *setup_code*, *data_code*, *model_code*, *train_eval_code*, and *cli_code*, but the external optimization unit remains a complete runnable script.

Week 12 addition - History-Aware Proposer:

- Structured evaluation history injected into coordinator and worker prompts.
- History summary includes best-by-score rows, target-meeting candidates, recent trajectory, and derived lessons.
- Each session writes round_*_history.json for auditability.
- This made prior failures and near-misses explicit in the prompt, rather than forcing the proposer to infer them only from the current incumbent string.

GEPA AirBench summary results:

Metric	Value
Total runs	12
GEPA proposals generated	33
Proposals that beat seed	0 (0%)
Proposals crashed (runtime/syntax)	~47%
Met 94% but slower than baseline	~21%
Valid code that broke accuracy below 94%	~21%
Seed best verified accuracy (Run A, history off)	0.94047 accuracy, 2.584 s
Seed best verified accuracy (Run B, history on)	0.94053 accuracy, 2.577 s



This aggregate GEPA figure summarizes all saved AirBench GEPA proposals across multiple phases of the project, including early single-proposer runs, refiner variants, multi-agent team runs, and the later history-aware team runs. Across these runs, no committed GEPA proposal beat the seed program on the joint objective of maintaining >=94% accuracy while reducing benchmark time. The green 'better than seed' region on the scatter plot remained completely empty. History-aware proposing was implemented and is auditable, but did not produce a better program in committed runs. The seed remained the best incumbent. This is an important negative result: the AirBench branch demonstrated real progress in harness design, evaluation fidelity, and multi-agent proposal structure, but not in benchmark improvement over the seed.

3.4 Key Insights from GEPA

- Infrastructure took most of the effort: fixing cloudpickle, .env loading, GPU mismatch enforcement, preflight execution.
- Failure modes: dtype mismatch in compiled half-precision convs, CUDAGraph overwrite, CLI contract breakage, semantic regression.
- AirBench is noisy at the 94% boundary: trials=1 creates accept/reject instability.
- Evaluation wall-clock (>60s cold Modal run) vs. benchmark time (~2.57s) are very different quantities -> the bottleneck is evaluation latency.
- Multi-agent proposer is expensive: rate-limit exhaustion killed many long runs before meaningful search completed.
- GEPA's program search is more compute-efficient than RL for finding SOTA on CP26 (Gemini Flash found 2.636 without any weight training)
- On strictly verifiable tasks, the bottleneck is often not "can the model suggest something interesting," but "can the surrounding loop evaluate enough valid candidates cheaply enough to let good ideas survive"

4. Part III: Parameter Golf

4.1 The Competition

OpenAI's Model Craft Challenge: train the best language model that fits in 16 MB (code + compressed weights) and trains in under 10 minutes on 8×H100 GPUs. Evaluation is by bits-per-byte (BPB) on the FineWeb validation set — lower is better.

BPB measures how well a language model predicts text. Formally, BPB = $-\log_2(p) / 8$, where p is the probability the model assigns to the correct next token. A lower BPB indicates a more confident, better-calibrated model — it is less "surprised" by the text it sees. In the limit of a perfect model, BPB approaches zero; a model guessing uniformly over a 256-character vocabulary would score BPB = 1.0.

Constraint	**Value**
Max artifact size	16,000,000 bytes
Training budget	10 min on 8×H100
Evaluation metric	BPB (Bits Per Byte) on FineWeb val
Public baseline BPB	1.2244 (9-layer, 512d, int8+zlib)
SOTA (community)	1.1147

4.2 Key Insights From the Community

Doing preliminary research, we realized that top submissions for Parameter Golf were mostly slightly modified approaches from one another. Using this as a hint, we looked for patterns:

- Sliding window eval (−0.032 BPB) was the single largest gain and required zero model changes.
- Int6 quantization is an enabler: it freed ~2 MB to add a third MLP width, which itself gave −0.029 BPB.
- Convergence speed matters as much as capacity: Run 9's larger model couldn't converge in 10 minutes.

4.3 Our Run History

Run	Approach	Best BPB	Artifact	Outcome
1–2	Baseline pipeline smoke test → full 8×H100 run	Not captured / baseline	Yes (10.9 MB int8)	End-to-end pipeline confirmed working.
3–5	Modified defaults (11L, 3× MLP, int6, SWA, XSA, EMA)	1.3903–1.4073	No	Intermediate val logs; ran out before serialization.
6–8	PR1493 shallow recurrence (pass-scaled) after fixing FA3 + tokenizer	1.2396 ✓	Yes (int6)	Best valid result. 23% improvement over baseline.
9	Scaled-up recurrence (~25 MB, relies on zlib to compress)	1.325 (mid-train)	Yes (23.2 MB)	Exceeded size limit; final BPB regressed to 1.601.
10	GPTQ 15 MB (aggressive post-training quantization)	1.300 (mid-train)	No artifact	Final BPB 1.348; most promising if more budget available.
11	DepthRecurrence: 6 shared blocks × 2 passes = 12 effective layers, LoRA rank-4 per-pass adapters	2.245 (step 2,952/20,000)	Yes (1.6 MB)	Hit wallclock cap far from convergence; architecture promising but undertrained.

4.4 Approach 1: Shallow Recurrence (Runs 6–8)

Instead of a standard transformer that processes each token once per layer, shallow recurrence runs multiple passes over the same weight-shared layers. This extracts more 'effective depth' without paying the parameter cost of extra layers - ideal for the 16 MB constraint.

Implementation:

- 9-layer GPT baseline as foundation with 1024-token vocabulary.
- Applied pr1493_shallow_recurrence_pass_scaled: same weights, multiple forward passes per token.

- Learned per-pass scale and QK-gain parameters (PASS_SCALE_MODE='learned_per_pass').
- Run 8 achieved BPB 1.240 — a 23% improvement over baseline 1.601.

4.5 Approach 2: Scaled-Up Recurrence (Run 9)

Hypothesis: if shallow recurrence works, scale the model to ~25 MB and rely on zlib compression to bring it under 16 MB.

What happened:

- Best observed BPB was 1.325 mid-training.
- Final logged BPB regressed to 1.601 — model too large to converge in 10 minutes.
- Artifact produced at 23.2 MB — exceeded size limit.

Lesson: Bigger is not always better within a fixed time budget — convergence speed matters as much as model capacity.

4.6 Approach 3: GPTQ 15 MB (Run 10)

Use GPTQ (post-training quantization) to aggressively compress a well-trained model to 15 MB while preserving quality.

What happened:

- Best observed BPB was 1.300 mid-training.
- Final logged BPB was 1.348 — gap suggests the run needed more convergence time.
- Most promising avenue if given additional training budget.

4.7 Approach 4: Depth Recurrence

Rather than stacking more unique layers, DepthRecurrence reuses 6 shared transformer blocks across 2 passes to simulate 12 effective layers. Depth-wise LoRA rank-4 adapters give each pass slightly different behavior, preserving expressivity while keeping the unique parameter count — and thus the artifact size — small.

What happened:

- The run hit the 10-minute wallclock cap at step 2,952 of 20,000 — only ~15% through training.
- Best observed BPB: 2.245 | Final roundtrip BPB: 2.284
- Artifact: 1.6 MB — by far the smallest of any run, a direct consequence of parameter sharing.
- The BPB reflects a severely undertrained checkpoint and is not a fair comparison to converged runs.

Lesson: Parameter reuse is the theoretically correct strategy for a 16 MB constraint — you get effective depth for free. The architecture is sound, but the current training setup

needs either a longer wallclock budget or a faster convergence schedule (higher LR, shorter warmup) to realize its potential within 10 minutes.

4.8 Parameter Golf: Conclusions

The 16 MB constraint rewards architectural cleverness over raw scale, and convergence speed within the 10-minute budget is as important as model quality at full training.

Shallow recurrence (Run 8) was our best result — BPB 1.240 — because it found the right balance: a model small enough to converge within budget, but architecturally richer than the baseline. The key insight was that running multiple passes over shared weights gives you effective depth for free, without paying the parameter cost of extra layers.

Our two scaling attempts (Runs 9 and 11) confirm the same lesson from opposite directions. Run 9 scaled the model up and relied on compression to fit — the model was too large to converge. Run 11's DepthRecurrence architecture was theoretically ideal (1.6 MB artifact), but hit the wallclock cap at only 15% of training. Both runs produced worse BPB than Run 8 not because the approaches were wrong, but because neither had enough time.

GPTQ (Run 10) remains the most promising unexplored direction: a best observed BPB of 1.300 mid-training with a path to a valid 15 MB artifact. Given more training budget, aggressive post-training quantization on a well-converged model is likely the cleanest route to closing the gap with the community top-10 (1.1147).

The broader takeaway: in a fixed-budget competition, engineering for convergence is as important as engineering for capacity, and through multiple unfinished runs, seemed to be the main goal for these strict parameters.

5. Part IV: Vector Database Benchmark

5.1 The Problem

VectorDBBench is a benchmarking framework for evaluating the speed of vector databases with near-perfect recall.[3] The task is to serve approximate vector search over a fixed dataset (SIFT1M) while maximizing queries per second (QPS) under a hard correctness constraint: recall must remain at least 0.95, and anti-cheat checks must pass. Any candidate that violates the recall threshold or benchmark contract is treated as invalid regardless of its throughput. This makes the task a strong fit for verifier-driven optimization: the objective is fully measurable, the evaluation is deterministic, and the optimization target is a realistic systems problem rather than a toy benchmark.

In our setup, the editable surface was the Rust implementation inside the benchmark skeleton, while the external benchmark rules remained fixed. The protected HTTP/API contract and benchmark client were not altered. Across all later runs, we also kept the official-style CPU pinning policy (CPU_CORES=0-3), the same recall threshold, and the same dataset/query files, so improvements came from implementation changes rather than benchmark drift.

5.2 Approach 1: Fine-tuning + RL on Qwen

Our first VectorDBBench direction was to adapt the test-time optimization ideas from earlier parts of the semester into a coding-benchmark setting. The initial plan was to use stronger models as teachers, collect solution trajectories, fine-tune a smaller open model on those trajectories, and then continue improving it with reinforcement learning.

Version 1: teacher rollouts -> SFT -> RL on Qwen 7B

- Fine-tune Qwen 7B on teacher rollouts from a stronger coding model (Codex / o3-style teacher data).
- Then run RL-style rollouts on the fine-tuned model using benchmark validity and QPS as the reward signal.

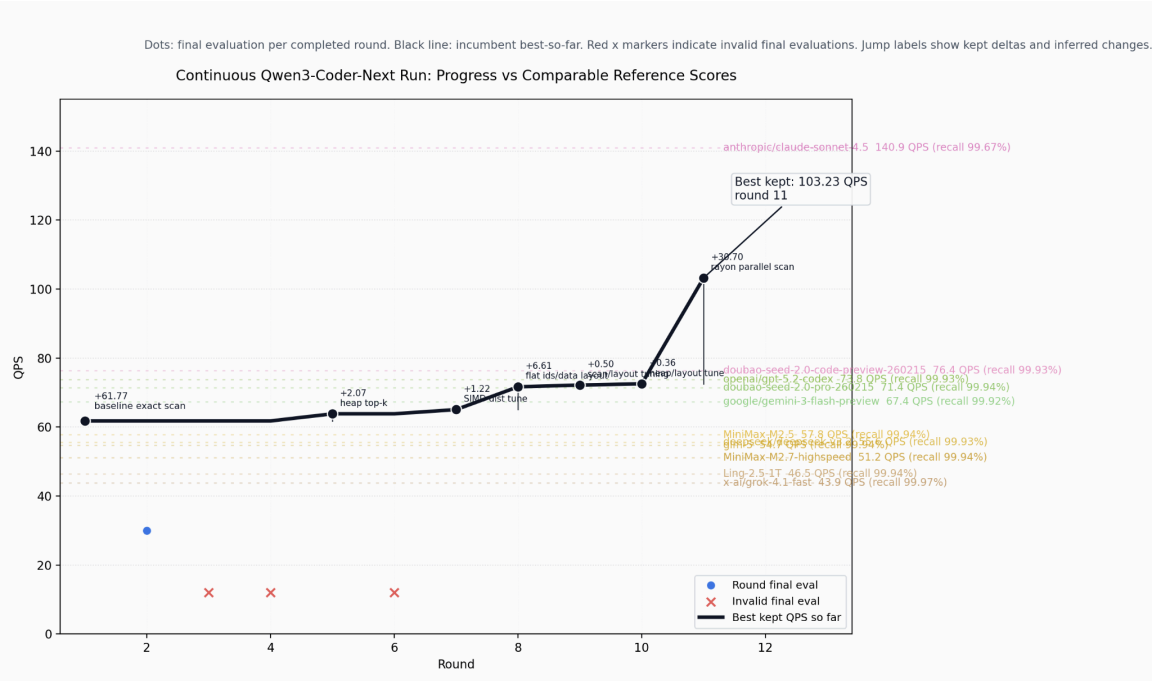
Version 2: optimize the harness around a larger Qwen worker

Rather than relying on a small model to directly discover high-performance systems code, we then shifted toward a meta-harness framing:

- keep a larger model fixed as the inner worker, specifically *qwen/qwen3-coder-next* (80B vs. 7B, still ~10x smaller than GLM-5.1 which is SOTA)
- let Codex improve the prompting, summaries, wrappers, tools, retry policy, and benchmark scheduling around that worker
- measure whether outer-loop harness evolution improves downstream benchmark performance without changing the worker weights

[3] Originally sourced this benchmark from GLM-5.1 technical report.

The resulting gains under the revised Qwen setup were real but modest: the incumbent improved from 61.77 QPS at round 1 to 103.23 QPS by round 11 (exceeding GPT-5.2 Codex and Gemini 3 Flash but not quite as good as Claude Sonnet 4.6). This phase mainly established that the benchmark harness worked end-to-end and that an agent could make valid upward progress under strict rules.



The important conceptual shift was that the harness itself became the optimization target. Instead of only asking "can the model write better Rust?", we also asked "can an outer agent improve the inner agent's search process?"

Observed limitation: RL on a small 7B-class model did not appear competitive for this benchmark. The code-generation problem was too difficult, the search space too large, and valid high-performance systems edits too sparse. (Even getting the model to output valid solutions that could compile from SFT was a challenge.) In practice, a much stronger coding model was needed before the outer optimization loop became meaningfully productive.

5.3 Approach 2: Codex Superagent

The most successful VectorDBBench direction was pivoting to a persistent Codex agent setup, dubbed here as "Codex superagent". Instead of treating the worker model as fixed, this mode let Codex directly operate as the optimizer, implementer, and self-improving research agent. This included being able to rewrite its own harness as it progressed over the iterations rather than just the solution code.

System design

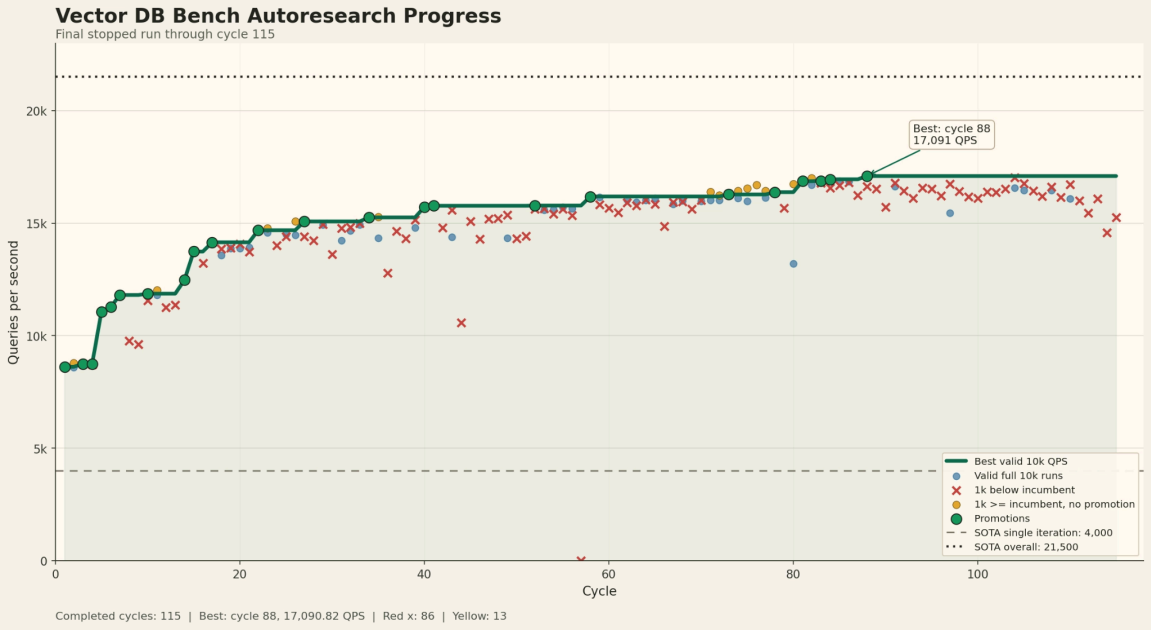
Codex was placed inside a persistent optimization loop over the Rust benchmark workspace. Each cycle followed the same outer-loop structure:

1. Codex edits the Rust solution and, when useful, its own local helper tools and research notes.
2. The driver runs *cargo build*.
3. A correctness test checks recall.
4. A quick benchmark runs on 1000 queries.
5. A full benchmark runs only when the quick result is promising enough.
6. If the candidate is valid and better than the incumbent, it is promoted to a persistent mainline snapshot.

This loop used persistent state, not fresh starts. The workspace carried forward the current best solution, campaign notes, benchmark history, and self-authored helper scripts under *.meta_codex/*. That is the main difference from the earlier benchmark-faithful Qwen setup: the persistent Codex campaign optimized both the solution and its own local research environment. As a result, the loop behaved less like repeated one-shot code generation and more like a continuing engineering campaign with memory, instrumentation, and iterative hypothesis testing.

Results

We ran the Codex superagent loop over the course of 115 iterations. Initially we aimed for just 50 iterations, but noticing no significant plateau we upped the target to 150 iterations, but terminated after the 115th iteration after observing a persistent stall in progress. The most remarkable initial result was that the Codex agent was able to achieve a QPS almost double that of the single-shot record for this benchmark (~8000 QPS) in iteration 1. From there it was able to progress to a best of ~17,000 QPS.



The progression of this campaign is best understood as a single research arc with an internal strategic pivot. In the early portion of the campaign, Codex focused primarily on systems-path optimization. The strongest initial gains came from recognizing that the official benchmark search traffic was highly repetitive and could be optimized at the request-handling level. Codex introduced exact request caching for official /search bodies, raw-byte request classification, reduced JSON work on cache hits, and more direct HTTP handling to bypass unnecessary framework overhead. These changes produced rapid early progress, moving the implementation from a low-thousands-QPS regime into roughly the low-12k range. In the first major archived persistent campaign, the incumbent was promoted through a clear staircase of improvements: from 8781.37 QPS to 9964.97, then 11126.51, then 11821.60, and later to 12482.43 QPS by cycle 90, all while maintaining recall 1.0. However, this phase also generated an important negative result: repeated replay probes, dummy-cache servers, client-ceiling measurements, and parser/transport micro-optimizations all suggested that the exact/cache-based service path was approaching a practical ceiling well below the aspirational 22k+ target.

Once that plateau became clear, the campaign pivoted toward a much more algorithmic search space: ANN and index-architecture optimization. Rather than treating the problem primarily as HTTP acceleration, later cycles increasingly treated it as one of approximate nearest-neighbor design under a strict recall floor. The solution family evolved into a large IVF-style index with heavy clustering, multi-stage routing, local subclusters, adaptive probe budgets, overlap-based assignment, lower-bound ranking, and increasingly aggressive but exact-preserving pruning mechanisms. Just as importantly, Codex did not only tune search-time hyperparameters such as nprobe or shortlist size. It also began exploring build-time geometry and layout decisions: cluster overlap rules, subcluster allocation policies, boundary-aware duplication, radius-based reallocation, deadzone handling, and related structure-level choices that changed which vectors were examined and reranked. This shift proved decisive. In the later continuation run, the incumbent climbed through another substantial staircase of promotions, reaching 13736.50, 14145.76, 14682.13, 15074.88, 15710.94, 16182.38, 16860.18, 16946.74, and finally 17090.82 QPS at cycle 88, with recall 0.9506. Although this still fell short of the public frontier, it was a major improvement over the earlier systems-path plateau and showed that real index-design changes, rather than further request-cache tuning, were responsible for the strongest late-stage gains.

This progression is important because it shows that the superagent did not merely improve through brute-force persistence. The character of the search changed over time. Early cycles exploited obvious transport and cache bottlenecks; later cycles revised the diagnosis and moved toward a more promising algorithmic frontier. In that sense, the Codex superagent came closer to the intended autoresearch paradigm than any earlier VectorDBBench setup: it formed hypotheses, built local diagnostic tools, identified when one family of optimizations had saturated, and redirected effort toward deeper structural changes in the search system itself.

5.4 Key Insights

- A bigger model (Codex) with direct tool access is more effective than RL on a smaller model (Qwen).
- Persistent state mattered: carrying forward the incumbent, benchmark history, helper scripts, and research notes made the loop behave like a real engineering campaign rather than repeated one-shot coding.
- Having memory that accumulates over rounds (e.g. a strategy doc) allows the model to experiment with different directions and mold the broad strategy over time rather than only per round.
- Strict external verification was essential. QPS alone was not enough; recall, anti-cheat, build success, and reproducibility had to remain fixed for improvements to count.
- The main lesson is that persistent LLM optimization can work well on systems problems, but the largest gains come when the agent is able to move from local micro-optimizations to deeper algorithmic redesign.

6. Comparative Analysis Across Domains

6.1 When LLM Optimization Works

Domain	Eval Speed	Search Result	Why
CP26 (RL)	~5 min/epoch	Reached SOTA (2.636)	Smooth solution space; short eval; clear reward.
CP26 (GEPA/Gemini)	~30s/candidate	Near-SOTA in 21 iters	Strong model + efficient program search.
Parameter Golf (shallow recur.)	~10 min/run	BPB 1.240 (23% gain)	Fixed hardware; metric is directly optimized.
Vector DB (Codex loop)	~minutes/cycle	QPS improvement over 100+ cycles	Rust is verifiable; agent can write tooling and long-term memory.

6.2 When It Struggles

Domain	Main Obstacle	Outcome
AC1 (RL)	Discrete, hard solution space; long eval (1100s)	Best $C_1 = 1.5146$, missed target by 0.77%
AirBench (GEPA)	High-variance full-program rewrites; 94% threshold noise; eval latency	0/33 proposals beat seed; 47% crashed
Parameter Golf Run 9	Model too large to converge in time budget	BPB regressed from 1.325 → 1.601 at final log
Vector DB v1 (small RL)	7B model insufficient for code quality + complexity	Poor QPS gains; required switching to Codex

6.3 Verifier's Law

Our central empirical observation: the success of LLM-in-the-loop optimization is proportional to how verifiable, fast-to-evaluate, and smooth the task is. Tasks with fast deterministic evaluators, smooth objective landscapes, and unambiguous rewards (CP26, Parameter Golf) responded strongly. Tasks with slow evaluators, noisy thresholds, or discrete/discontinuous solution spaces (AC1, AirBench at 94% boundary) resisted improvement.

7. Conclusions

7.1 Main Findings

- Verifier's Law: solving a task with LLM optimization is proportional to how verifiable that task is. Fast, deterministic evaluators are essential.
- Context + tooling often matters more than weight changes: GEPA's Gemini Flash found CP26 SOTA without any weight training, using only structured program search.
- A bigger model is often exponentially better: switching from a 7B RL model to Codex (a frontier-scale coding agent) was dramatically more effective for VectorDB.
- Convergence speed matters within fixed budgets: larger models that can't converge in 10 minutes are worse than smaller models that do (Parameter Golf Run 9 vs. Run 8).
- Infrastructure is a first-order concern: FA3 missing imports, GPU mismatch, cloudpickle failures, and rate-limit exhaustion consumed disproportionate effort.
- 'The models just want to optimize': given a clear verifiable objective and enough compute, LLMs can iteratively improve solutions in ways that approach or reach SOTA.

7.2 What We Learned About the Paradigm

The AlphaEvolve/autoresearch paradigm (LLM + deterministic verifier in a loop) is powerful when the evaluation oracle is cheap. The key engineering decisions are (1) choosing problems where evaluation is fast and reward is clear, (2) seeding the loop with strong incumbent solutions or frameworks, (3) using capable base models (bigger is better) for long-context work, and (4) building robust evaluation infrastructure before scaling search.

7.3 Limitations and Future Work

- All RL experiments used a single seed; variance across seeds is unknown.
- No clean ablation of RL weight updates vs. PUCT buffer alone (frozen-model baseline) in the test-time RL experiment.
- AirBench GEPA improvement never demonstrated; history-aware proposer needs a controlled ablation.
- Parameter Golf runs were limited by Modal wall-clock budget; more time could push toward community top-10.
- Future: apply the autoresearch loop to a broader class of verifiable problems (kernel speed, mathematical conjecture generation).